# SQL

Structured Query Language

# Tema 4
# DQL
# (Data Query Language)

**SELECT es la instrucción principal de SQL y permite realizar consultas a la base de datos.**

- Es muy potente y flexible
- Consulta una o varias tablas
- Filtra filas y columnas
- Ordena resultados
- Agrupa datos (SUM, COUNT, etc.)
- Puede generar tablas nuevas a partir de consultas

# Tipos de Consultas SELECT

## Consulta Básica

- Sentencia básica con única tabla en el FROM

## Consultas Avanzadas

- **Combinación de tablas:**
  - PRODUCTO CARTESIANO (CROSS JOIN)
  - INNER JOIN
  - NATURAL JOIN
  - OUTER JOIN
- **Subconsultas:**
  - Sintaxis en el FROM
  - Sintaxis en el WHERE o HAVING
  - Sintaxis en el SELECT
- **Operaciones de Conjuntos:**
  - UNION
  - INTERSECT
  - EXCEPT (DIFERENCIA)

# Consultas Avanzadas

# Combinación de Tablas

## ¿Por qué combinar tablas?

- Para poder obtener información relacionada entre distintas tablas de una base de datos: p.e. comarcas–poblaciones, clientes–facturas, etc.

## ¿Qué tipos de combinaciones podemos hacer?

- Producto cartesiano (CROSS JOIN) - NO deseado
- Combinación interna (INNER JOIN) – la habitual
- Combinación externa (LEFT/RIGHT OUTER JOIN), muy útil en ciertos casos

## Consideraciones especiales

- Utilizar Alias de tabla: necesarios cuando una misma tabla aparece dos veces.
- Para combinar n tablas, se necesitan n−1 condiciones de unión.
- Si falta una unión, aparecerá el producto cartesiano.

# Producto Cartesiano (CROSS JOIN)

```
SELECT ...
 FROM tabla1, tabla2,..
```

## ¿Cómo funciona?

- Combina cada fila de una tabla con cada fila de otra tabla, sin condiciones de unión, generando un resultado con todas las combinaciones posibles

## Resultado de ejecución:

- En general **no lo queremos**, porque da resultados incorrectos; lo habitual es combinar las tablas adecuadamente mediante un JOIN.

| | |
|---|---|
| Ejemplo: | SELECT c.nom_c, nom<br>FROM COMARQUES c , POBLACIONS p; |
| Resultado: | Al ejecutarla obtenemos **18.428 filas** (34 x 542), porque se combina cada comarca con todos los pueblos, pertenezcan o no a ella. |
| Nota: | Como ambas tablas tienen un campo con el mismo nombre (nom_c), debemos poner el nombre de la tabla también para evitar ambigüedades. |
| Optimizarla: | Sería combinar *cada comarca únicamente con sus poblaciones.* |

# Combinación INTERNA
# **INNER JOIN**

```
SELECT ...
  FROM tabla1 INNER JOIN tabla2
  ON condición
```

## ¿Cómo funciona?

- Combinar tablas de manera que dos campos (uno de cada tabla) coincidan.
- **Normalmente se trata de claves principales y externas. (Integridad Referencial)**

## Resultado de ejecución:

- Devuelve solo las filas relacionadas entre ambas tablas.

Ejemplo:

```
SELECT c.nom_c, p.nom
FROM COMARQUES c INNER JOIN
POBLACIONS p
ON c.nom_c = p.nom_c
```

Resultado:

Esto devuelve sólo las filas donde coinciden los valores.

En este caso 542 filas (tantas como poblaciones)

# Alternativas a **INNER JOIN**

```
SELECT p.nom,i.nom
FROM POBLACIONS p INNER JOIN INSTITUTS i
ON p.cod_m = i.cod_m;
```

## WHERE

- Alternativa equivalente. Donde utilizamos un CROSS JOIN con la condición de la unión entre las tablas
- `FROM tabla1 , tabla2`
  `WHERE tabla1.campo = tabla2.campo`

## USING(campo)

- Solamente se pude usar si el campo se llama igual en ambas tablas.
- `FROM tabla1 INNER JOIN tabla2`
  `USING (campo)`

## NATURAL JOIN

- Combina por todos los campos con el mismo nombre (hay que usarlo con cuidado).
- `FROM tabla1 NATURAL JOIN tabla2`

**Ejemplos SIMILARES:** Muestra las poblaciones y sus institutos que coincidan en cod_m.

```
SELECT p.nom,i.nom
FROM POBLACIONS p, INSTITUTS i
WHERE p.cod_m = i.cod_m;
```

```
SELECT p.nom,i.nom
FROM POBLACIONS p INNER JOIN INSTITUTS i
USING(cod_m);
```

```
SELECT p.nom,i.nom
FROM POBLACIONS p NATURAL JOIN INSTITUTS
i;
```

# Combinación EXTERNA
# LEFT/RIGHT [OUTER] JOIN

```
SELECT ...
  FROM tabla1 LEFT/RIGHT [OUTER]
  JOIN tabla2
  ON condición
```

## ¿Cómo funciona?

- La combinación externa permite obtener todas las filas de una tabla, incluso aquellas que no tienen coincidencias en la otra tabla.
- **Resuelve el problema:** En una combinación interna (INNER JOIN), las filas sin coincidencia no aparecen. Con OUTER JOIN sí se incluyen.

## Tipos:

- **LEFT OUTER JOIN**: Devuelve todas las filas de la tabla izquierda y las coincidentes de la derecha.
- **RIGHT OUTER JOIN**: Devuelve todas las filas de la tabla derecha y las coincidentes de la izquierda.

## Importante:

- **Para evitar valores NULL**: en OUTER JOIN pueden utilizar funciones como **COALESCE()** ayudan a mejorar el resultado.

**Ejemplo**: muestra todas las poblaciones, incluso las que no tienen instituto (con valores NULL).

```
SELECT p.nom,i.nom
FROM POBLACIONS p LEFT JOIN INSTITUTS i
ON p.cod_m = i.cod_m ;
```

**Ejemplo**: Obtener poblaciones que no tengan instituto.

```
SELECT p.nom
FROM POBLACIONS p LEFT JOIN INSTITUTS i
ON p.cod_m = i.cod_m
WHERE i.codi IS NULL;
```

**Ejemplo:** Contar institutos por población

```
SELECT p.nom, COUNT(i.codi)
FROM POBLACIONS p LEFT JOIN INSTITUTS i
ON p.cod_m = i.cod_m
GROUP BY p.nom;
```

Otros ejemplos de uso:

- Detectar elementos sin relación (por ejemplo, artículos sin ventas).
- Obtener totales incluyendo valores nulos.

# Subconsultas

## ¿Para qué sirve?

- Una subconsulta es una consulta dentro de otra consulta.
- Una subconsulta puede contener todos los elementos que hemos visto hasta ahora.

## ¿Cómo se utiliza?

- Si va en el propio **SELECT**, normalmente sirve para obtener un valor global que no depende del resto de la consulta.
- Dentro de la cláusula en **FROM**, actúa como origen de datos y se ejecuta antes que la consulta principal.
- Si va en **WHERE** o **HAVING**, forma parte de una condición y permite comparar campos con valores devueltos por la subconsulta (incluyendo operadores especiales).

## Importante:

- Una subconsulta siempre va entre paréntesis.
- Es recomendable diseñar la consulta desde dentro hacia fuera: primero comprobar que la subconsulta funciona correctamente, y después construir la principal.

# Subconsulta en el **SELECT**

```
SELECT (Subconsulta)
  FROM tabla
```

Ejemplo: Obtiene la diferencia entre la altura de cada población y la media.

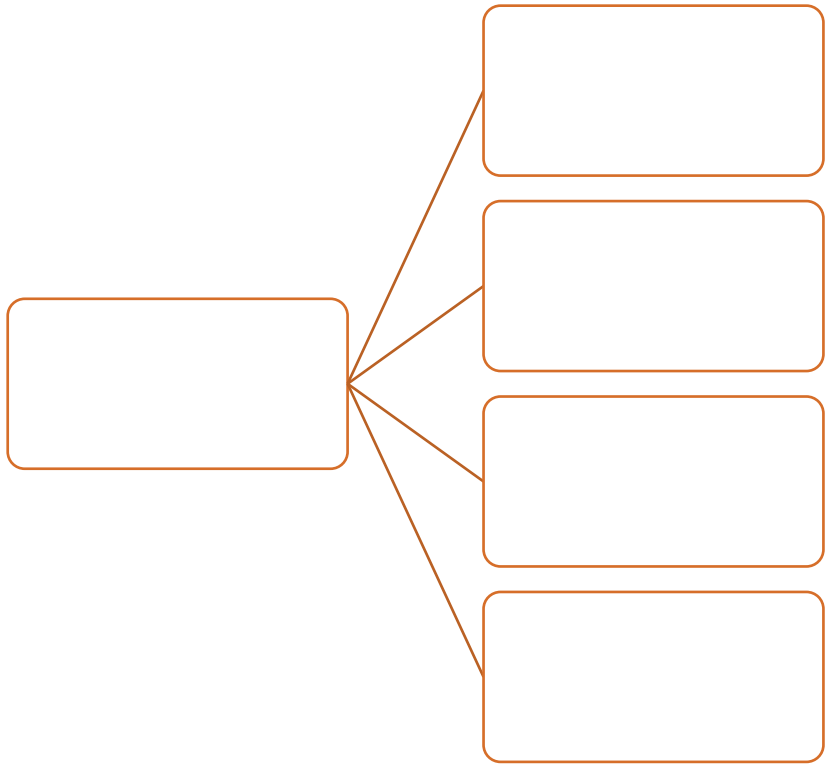
```
SELECT nom, altura, altura - (SELECT
AVG(altura) FROM POBLACIONS)
FROM POBLACIONS
```

Ejemplo: Sacar el nombre de la comarca con la provincia, número de pueblos de cada una y el porcentaje que supone respecto al total de pueblos.

```
SELECT COMARQUES.nom_c, provincia,
COUNT(cod_m),   COUNT(cod_m)*100.0 / (SELECT
COUNT(*) FROM POBLACIONES)
FROM COMARQUES INNER JOIN POBLACIONES
USING(nom_c) GROUP BY 1, 2;
```

# Subconsulta en el **FROM**

```
SELECT ...
FROM (Subconsulta) AS nomsubconsulta
```



Ejemplo: Obtiene la media de pueblos por comarca

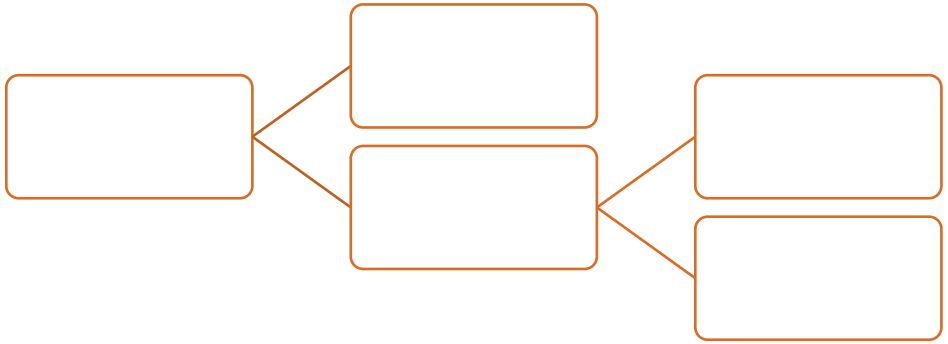
```
SELECT AVG(quants)
FROM  (SELECT COUNT(*) AS quants
FROM POBLACIONS GROUP BY nom_c) AS
subconsulta;
```

Ejemplo: Sacar la altura media de la comarca más grande y de la más pequeña.

```
SELECT MAX(mitjana), MIN(mitjana)
FROM (SELECT AVG(altura) AS
mitjana  FROM POBLACIONES GROUP BY
nom_c) AS subconsulta;
```

# Subconsulta en el **WHERE o HAVING**

```
SELECT ...
  FROM tabla
  WHERE  campo operador (Subconsulta)
```



**Ejemplo**: Obtiene toda la información de las poblaciones cuya altura es mayor a la media.

```
SELECT *
FROM POBLACIONS
WHERE altura > (SELECT AVG(altura) FROM POBLACIONS)
```

**Ejemplo**: Sacar toda la información de la población más alta y de la más baja.

```
 SELECT * FROM POBLACIONS
WHERE altura = (SELECT MAX(altura) FROM POBLACIONS)
OR altura = (SELECT MIN(altura) FROM POBLACIONS);
```

Ejemplo: Sacar la población más alta de cada comarca.

```
SELECT * FROM POBLACIONS T1
WHERE altura = (SELECT MAX(altura) FROM POBLACIONS WHERE
nom_c = T1.nom_c);
```

**Ejemplo**: Obtener el nombre de la comarca y la provincia de las comarcas que tienen una altura media más alta que la media de todas las poblaciones

```
SELECT c.nom_c, provincia, AVG(altura) as "altura media"
FROM COMARQUES c, poblacions p WHERE c.nom_c = p.nom_c
GROUP BY c.nom_c, provincia
HAVING AVG(altura) > (SELECT AVG(p2.altura) FROM
POBLACIONS p2);
```

# Predicados
# ALL, ANY/SOME

```
SELECT ...
  FROM tabla
  WHERE campo operador predicadoA/A (Subconsulta)
```

**PROBLEMA:**

- **Si la subconsulta devuelve más de un valor,** no se puede comparar un campo con varios valores.
  **SOLUCIÓN:** Utilizar predicados ANY, ALL/SOME

**Predicado ALL**

- La comparación debe cumplirse para *todos* los valores devueltos en la subconsulta.

**Predicado ANY / SOME**

- Basta que se cumpla para *alguno* de los valores de la subconsulta.

Ejemplo:
Obtiene toda la información de las poblaciones cuya altura coincide con alguna de las alturas máximas, aunque no sean de su comarca (necesita ajustes).

```
SELECT *   FROM POBLACIONS
WHERE altura = ANY (SELECT
MAX(altura) FROM POBLACIONS
GROUP BY nom_c)
```

# Operadores **IN, EXIST**

```
SELECT ...
  FROM tabla
  WHERE  campo operadorI/E (Subconsulta)
```

**[NOT] IN:**

- Comprueba si un valor pertenece o no a un conjunto devuelto por la subconsulta.

**[NOT] EXISTS:**

- Comprueba si la subconsulta devuelve filas.

Ejemplo:
Obtiene toda la información de las poblaciones sin instituto.

```
SELECT * FROM POBLACIONS
WHERE cod_m NOT IN (SELECT cod_m
FROM INSTITUTS);
```

**Ejemplo**:
Obtiene toda la información de las poblaciones sin instituto.

```
SELECT * FROM POBLACIONS
WHERE NOT EXISTS (SELECT *  FROM
INSTITUTS WHERE cod_m =
POBLACIONS.cod_m);
```

**Ejemplo**: Sacar toda la información de las poblaciones que tienen más de 5 institutos.

```
SELECT * FROM POBLACIONES WHERE
cod_m IN (SELECT cod_m FROM
INSTITUTS GROUP BY cod_m HAVING
COUNT(*) > 5);
```

# Consulta con Operadores de Conjuntos

## ¿Para qué sirve?

- Las **operaciones de conjuntos en SQL** sirven para *combinar y comparar resultados* de diferentes consultas.

## ¿Cómo se utiliza?

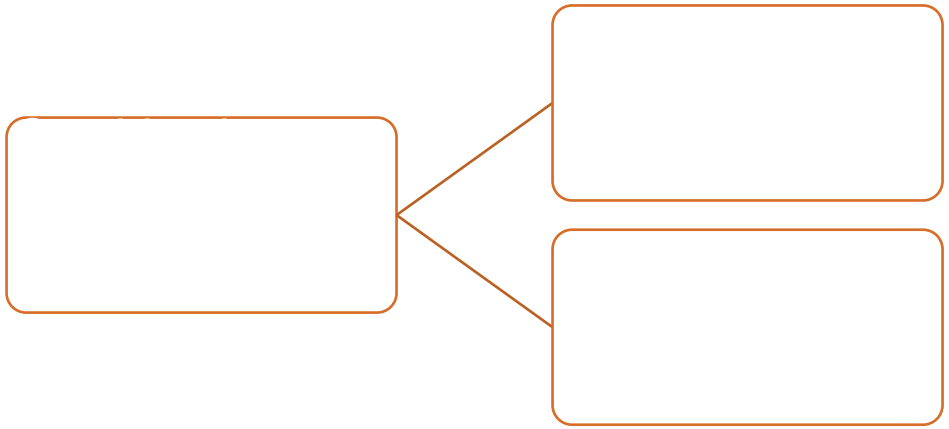
- **UNIÓN**: Combina los resultados de dos o más consultas.
- **INTERSECCIÓN**: Devuelve únicamente las filas que aparecen en **todas** las consultas.
- **DIFERENCIA**: Muestra las filas de la **primera consulta** que no aparecen en la segunda.

## Importante:

- Todas consultas deben devolver igual número de columnas.
- Todas consultas deben  devolver el mismo tipo de datos o  compatibles.
- Las consultas pueden ser tablas u otras sentencias SQL.
- Los nombres de los campos estarán determinados por la primera consulta.
- Para ordenar el resultado, ponemos ORDER BY al final de la última consulta, pero refiriéndose siempre a los campos de la primera consulta.

# Operación Conjunto **UNION [ALL]**

```
[TABLE] consulta1
UNION [ALL]
[TABLE] consulta2
[ORDER BY]
```



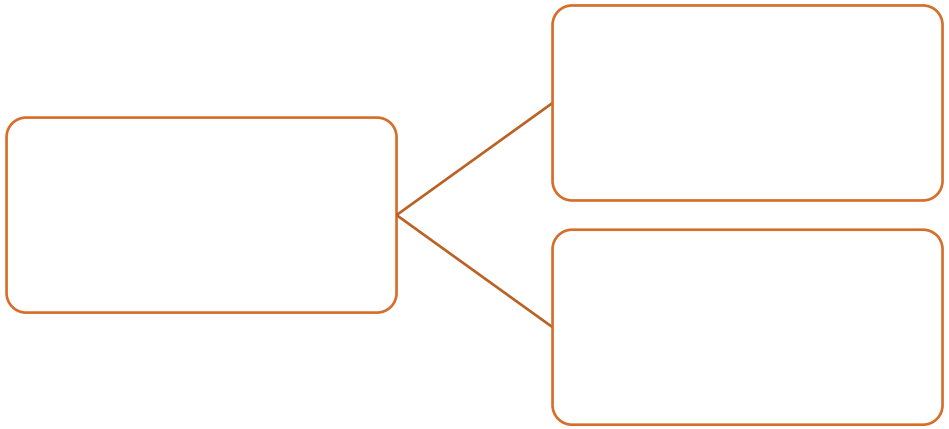
**Ejemplo**: Queremos ver en un único resultado tanto el nombre de las comarcas como el nombre de las poblaciones, siempre con el nombre de la provincia al lado .

```
SELECT nom_c, provincia FROM COMARQUES
UNION
SELECT nom, provincia FROM COMARQUES
INNER JOIN POBLACIONS USING (nom_c)
ORDER BY nom_c;
```

*Como curiosidad, **saldrán 575 filas**, pero si usáramos UNION ALL saldrían 576.*
*Ya que la comarca de la ciudad de Valencia se llama Valencia y está en la provincia de Valencia. Por tanto, es una fila que aparece tanto en la primera como en la segunda consulta.*

# Operación Conjunto **INTERSECT [ALL]**

```
[TABLE] consulta1
INTERSECT [ALL]
[TABLE] consulta2
[ORDER BY]
```

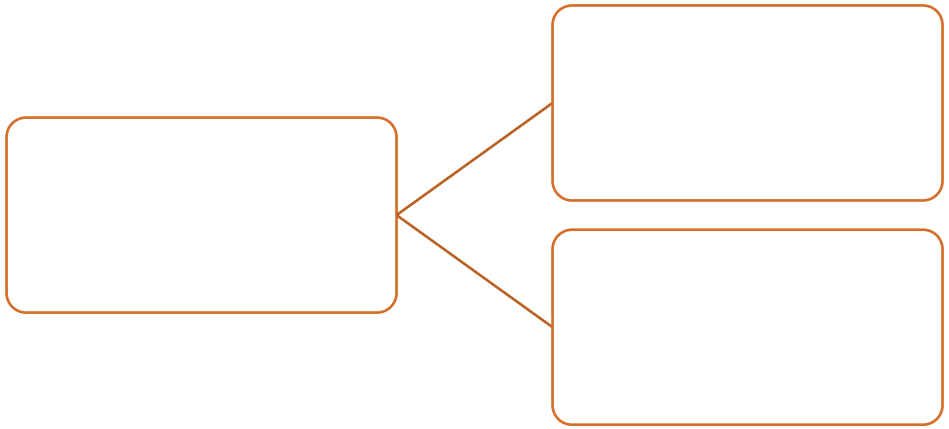


**Ejemplo**: Como en el ejemplo de la unión vimos que la fila Valencia - Valencia salía en ambas consultas, vamos a comprobar que aparece en la intersección:

```
SELECT nom_c, provincia FROM
COMARQUES
INTERSECT
SELECT nom, provincia FROM
COMARQUES INNER JOIN POBLACIONS
USING (nom_c)
ORDER BY nom_c;
```

# Operación Conjunto **EXCEPT [ALL]**

```
[TABLE] consulta1
EXCEPT [ALL]
[TABLE] consulta2
[ORDER BY]
```



**Ejemplo**: Usamos el mismo ejemplo anterior para comprobar que con EXCEPT no aparecerá la comarca Valencia, ya que hay una fila idéntica en la segunda consulta:

```
SELECT nom_c, provincia FROM
COMARQUES
EXCEPT
SELECT nom, provincia FROM
COMARQUES INNER JOIN POBLACIONS
USING (nom_c)
ORDER BY nom_c;
```

# FIN